

Zenith Future Skills Hub - Platform V1 Requirements

Document: Software Requirements Specification (SRS)

Product: Zenith Platform

Version: 1.0

Status: V1 Baseline

Date: 8 September 2026

Owner: Zenith Future Skills Hub

Scope distinction: This document defines the requirements for **Zenith Platform**, the software application. It does not define the Zenith Training curriculum, lessons, assessments or learner training content.

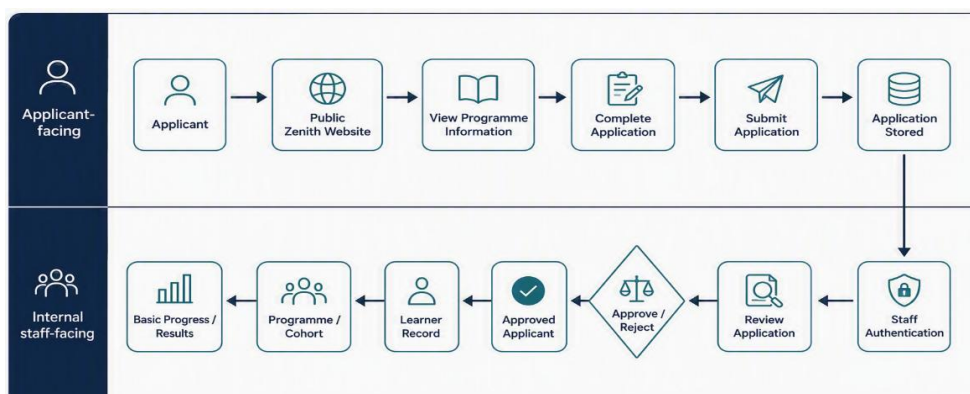
1. Purpose

Zenith Platform is a small web-based platform for **Zenith Future Skills Hub**, a South African nonprofit organisation providing technology skills programmes to young people.

The platform will support the organisation's basic digital process:

Inform → Apply → Review → Approve → Manage Learners → Track Progress

Figure 1: High Level User Flow



The purpose of V1 is to replace a largely manual application and learner-management process with a simple, secure and maintainable web application.

2. Problem Statement

Zenith needs a central platform through which prospective learners can:

- learn about Zenith and its programmes;
- submit an application; and
- receive confirmation of their application.

Authorised Zenith staff need to:

- view submitted applications;
- review applications;
- approve or reject applicants;
- manage approved learners; and
- record basic programme progress and results.

The V1 platform should provide these capabilities without introducing unnecessary enterprise-level complexity.

3. Goals

Zenith Platform V1 shall:

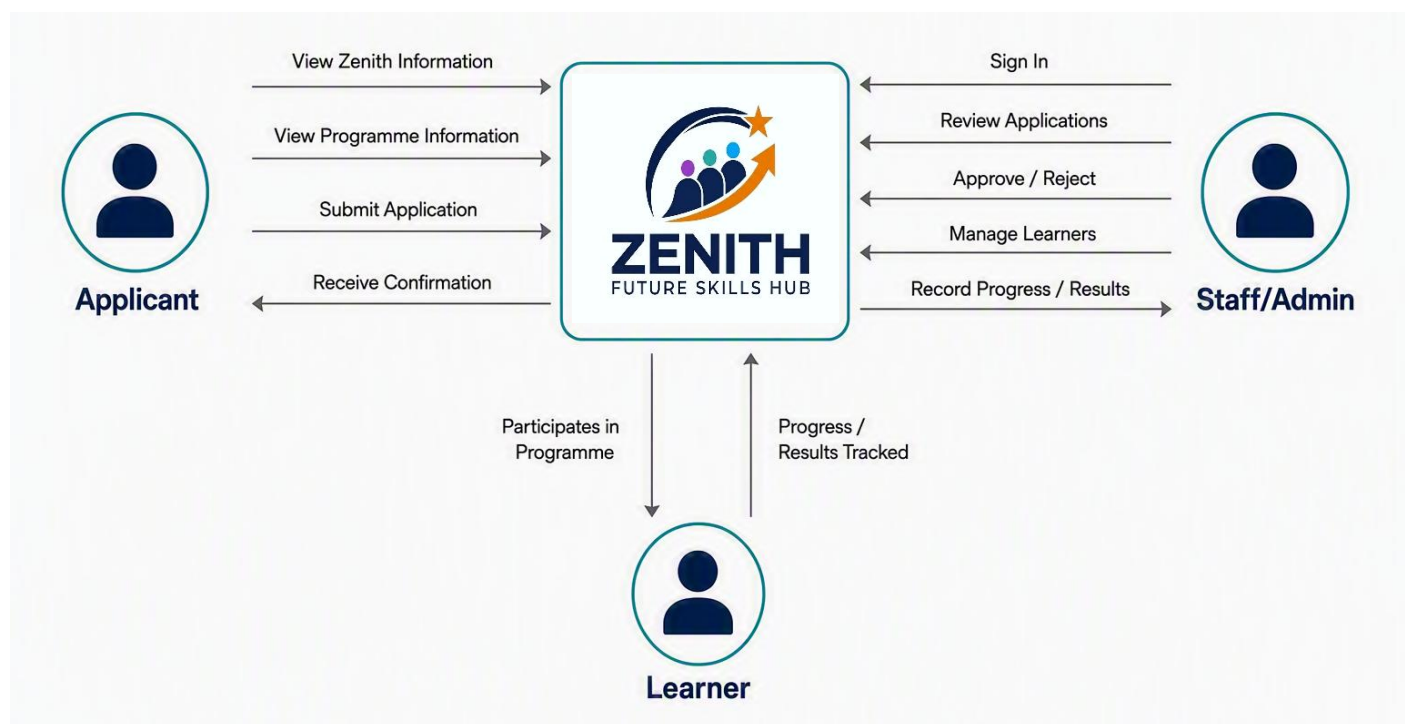
1. Provide a public website for Zenith.
2. Present programme information clearly.
3. Allow prospective learners to submit applications online.
4. Store applications in a database.
5. Allow authorised staff to review applications.
6. Allow staff to approve or reject applications.
7. Allow approved applicants to be managed as learners.
8. Associate learners with programmes and cohorts.
9. Support basic progress and result tracking.
10. Protect application and learner information.
11. Demonstrate a complete full-stack solution from requirements through deployment.

4. Stakeholders and Actors

Actor	Description
Applicant	Person interested in applying for a Zenith programme
Staff/Admin	Authorised Zenith user responsible for applications and learners
Learner	Approved applicant participating in a programme
System	Platform services responsible for storing and processing information

For V1, **Staff/Admin can be treated as one internal user role** unless requirements later justify separate roles.

Figure 2: System Context Diagram



5. User Stories

Applicant

US-01

As an applicant, I want to learn about Zenith so that I can understand what the organisation offers.

US-02

As an applicant, I want to view programme information so that I can determine whether a programme is suitable for me.

US-03

As an applicant, I want to submit an online application so that I can apply for a programme.

US-04

As an applicant, I want confirmation that my application was submitted so that I know it was received.

Staff/Admin

US-05

As a staff member, I want to securely sign in so that only authorised users can access internal information.

US-06

As a staff member, I want to view submitted applications so that I can manage applicants.

US-07

As a staff member, I want to view an application's details so that I can assess the applicant.

US-08

As a staff member, I want to approve or reject an application so that applicants can be processed.

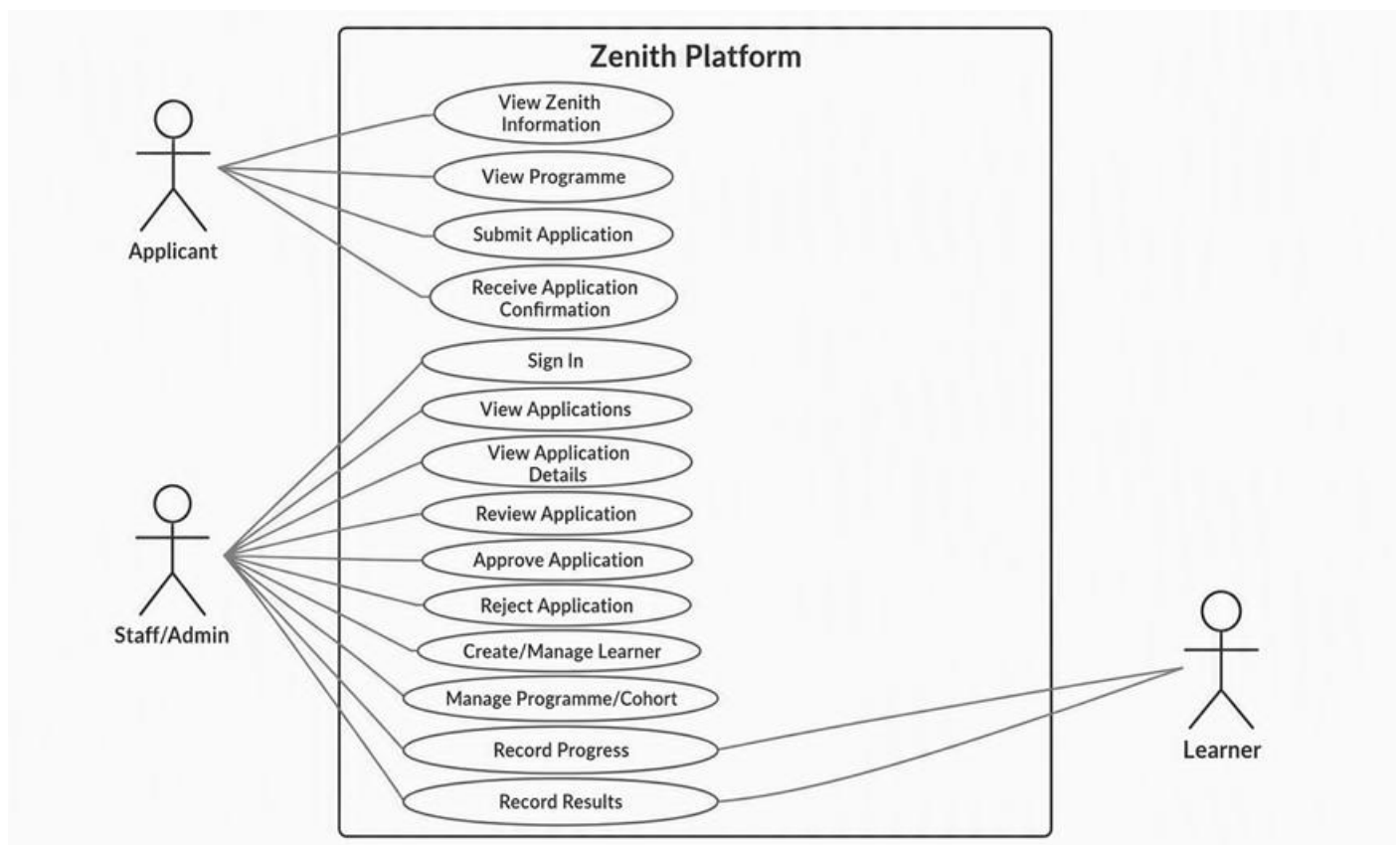
US-09

As a staff member, I want an approved applicant to become a learner so that I can manage programme participation.

US-10

As a staff member, I want to view and record basic learner progress/results so that learner participation can be tracked.

Figure 3: Use Case Diagram



6. Functional Requirements

6.1 Public Website

ID	Requirement	Priority
FR-01	The system shall provide a public home page.	Must
FR-02	The system shall provide information about Zenith and its programmes.	Must
FR-03	The system shall provide navigation between public pages.	Must
FR-04	The system shall provide an application entry point.	Must
FR-05	The website should maintain a consistent and usable visual design.	Should

Initial public pages

- Home
- About
- Programmes
- Programme Details
- Apply
- Contact

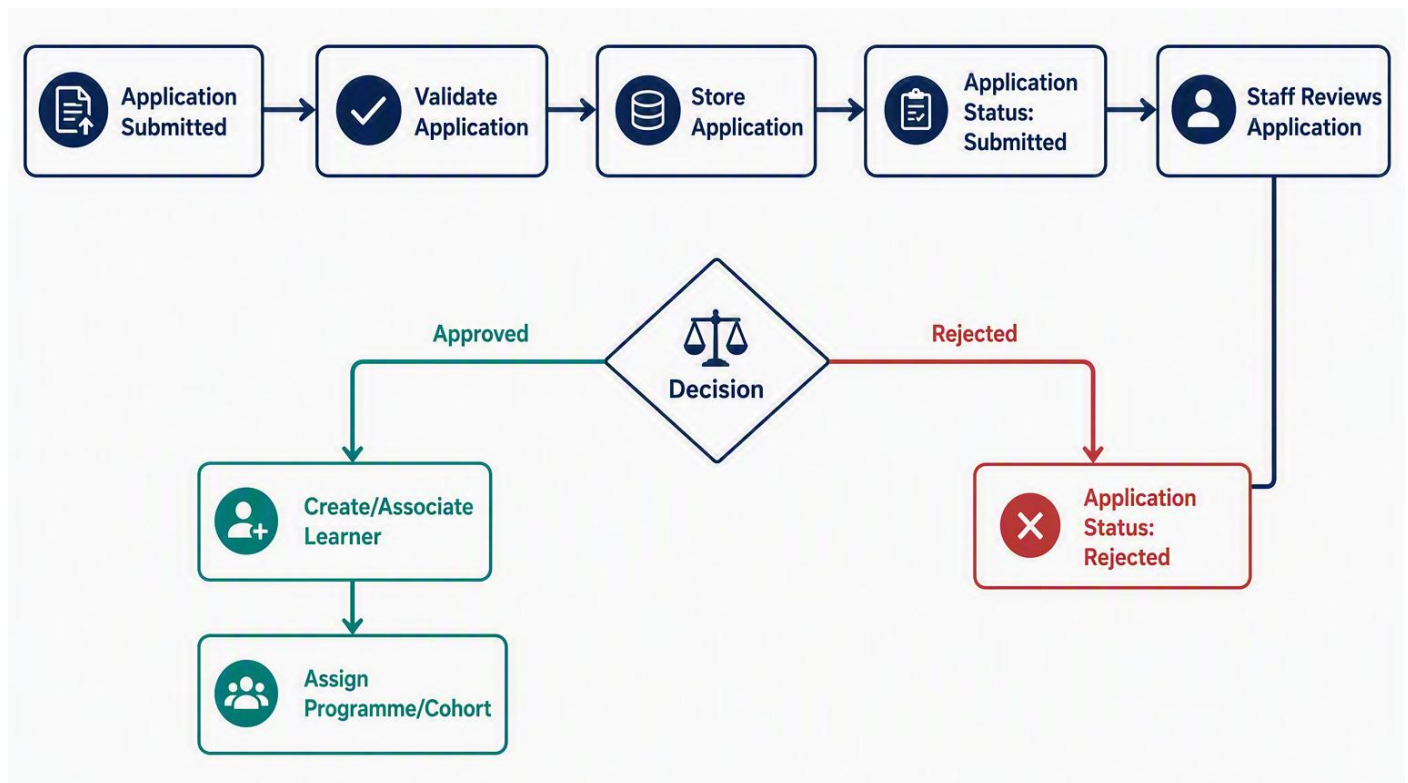
6.2 Application

ID	Requirement	Priority
FR-06	The system shall allow an applicant to submit an online application.	Must
FR-07	The system shall validate required application information.	Must
FR-08	The system shall store submitted applications in the database.	Must
FR-09	The system shall assign an application status.	Must
FR-10	The system shall prevent incomplete or invalid application data from being stored.	Must

Initial application status

- Submitted
- Under Review
- Approved
- Rejected

Figure 4 : Application Processing Flow



6.3 Authentication and Authorisation

ID	Requirement	Priority
FR-11	The system shall authenticate authorised internal users.	Must
FR-12	The system shall restrict protected application and learner functions to authorised users.	Must
FR-13	The system shall enforce access permissions for protected resources.	Must
FR-14	The system shall not expose protected learner/application information through public pages or unauthorised API access.	Must

6.4 Application Review

ID	Requirement	Priority
FR-15	Staff shall be able to view submitted applications.	Must
FR-16	Staff shall be able to view application details.	Must
FR-17	Staff shall be able to approve or reject applications.	Must
FR-18	The system shall retain the current application status.	Must

6.5 Learner Management

ID	Requirement	Priority
FR-19	Staff shall be able to create or associate a learner record for an approved applicant.	Must
FR-20	Staff shall be able to view learner records.	Must
FR-21	A learner shall be associated with a programme and, where applicable, a cohort.	Must
FR-22	The system should support recording basic learner/module progress.	Should
FR-23	The system should support recording basic learner results.	Should

The learner management functionality should remain **basic**. It is not intended to become a full Learning Management System.

6.6 Programme and Data Management

ID	Requirement	Priority
FR-24	The system shall store programme information required by the platform.	Must
FR-25	The system shall maintain relationships between relevant applicants, applications, learners, programmes, cohorts, modules and results.	Must
FR-26	The database shall enforce appropriate data integrity constraints.	Must

7. Non-Functional Requirements

ID	Requirement
NFR-01 Security	The system shall protect authenticated and sensitive application/learner information.
NFR-02 Privacy	Personal information shall be handled with appropriate POPIA considerations.
NFR-03 Usability	The platform should be simple and understandable for applicants and staff.
NFR-04 Performance	The system should provide reasonable response times for a small organisation and normal expected usage.
NFR-05 Maintainability	Frontend, backend and database responsibilities should be clearly separated and maintainable.
NFR-06 Reliability	The system should handle common errors without exposing sensitive information or crashing unexpectedly.
NFR-07 Accessibility	The interface should use semantic HTML, appropriate labels and readable content, with keyboard accessibility where practical.
NFR-08 Scalability	The architecture should allow reasonable future growth without introducing unnecessary complexity in V1.
NFR-09 Observability	Appropriate application logging should be available for troubleshooting without unnecessarily logging sensitive personal information.

8. V1 Scope

Figure 5: V1 Scope

IN SCOPE	OUT OF SCOPE
✓ Public website ✓ Programme information ✓ Applications ✓ Application review ✓ Approval/rejection ✓ Authentication ✓ Learner management ✓ Programme/cohort ✓ Basic progress/results ✓ SQL database ✓ API ✓ Testing ✓ Cloud deployment	✗ Full LMS ✗ Payments ✗ Finance ✗ HR ✗ Mobile app ✗ Real-time chat ✗ Advanced analytics ✗ AI admissions ✗ Microservices ✗ Complex enterprise infrastructure

In Scope

Public

- Home
- About
- Programmes
- Programme details
- Application
- Contact
- Application confirmation

Internal

- Authentication
- Application list
- Application details
- Application review
- Approve/reject
- Learner records
- Programme/cohort information
- Basic progress

- Basic results

Technical

- Frontend
- Backend/API
- SQL database
- Authentication/authorisation
- Testing
- Cloud deployment
- GitHub source control and documentation

Out of Scope

The following are deliberately excluded from V1:

- Full Learning Management System
- Online payments
- Finance/accounting
- HR management
- Mobile application
- Real-time chat
- Advanced analytics/BI
- AI-based admissions decisions
- Complex notification infrastructure
- Microservices
- Complex multi-tenant architecture
- Enterprise-scale infrastructure
- Sophisticated role hierarchy

These may be considered in future versions if there is a genuine business need.

10. Initial Domain Model

The initial domain should consider the following entities:

Applicant

Application

User

Learner

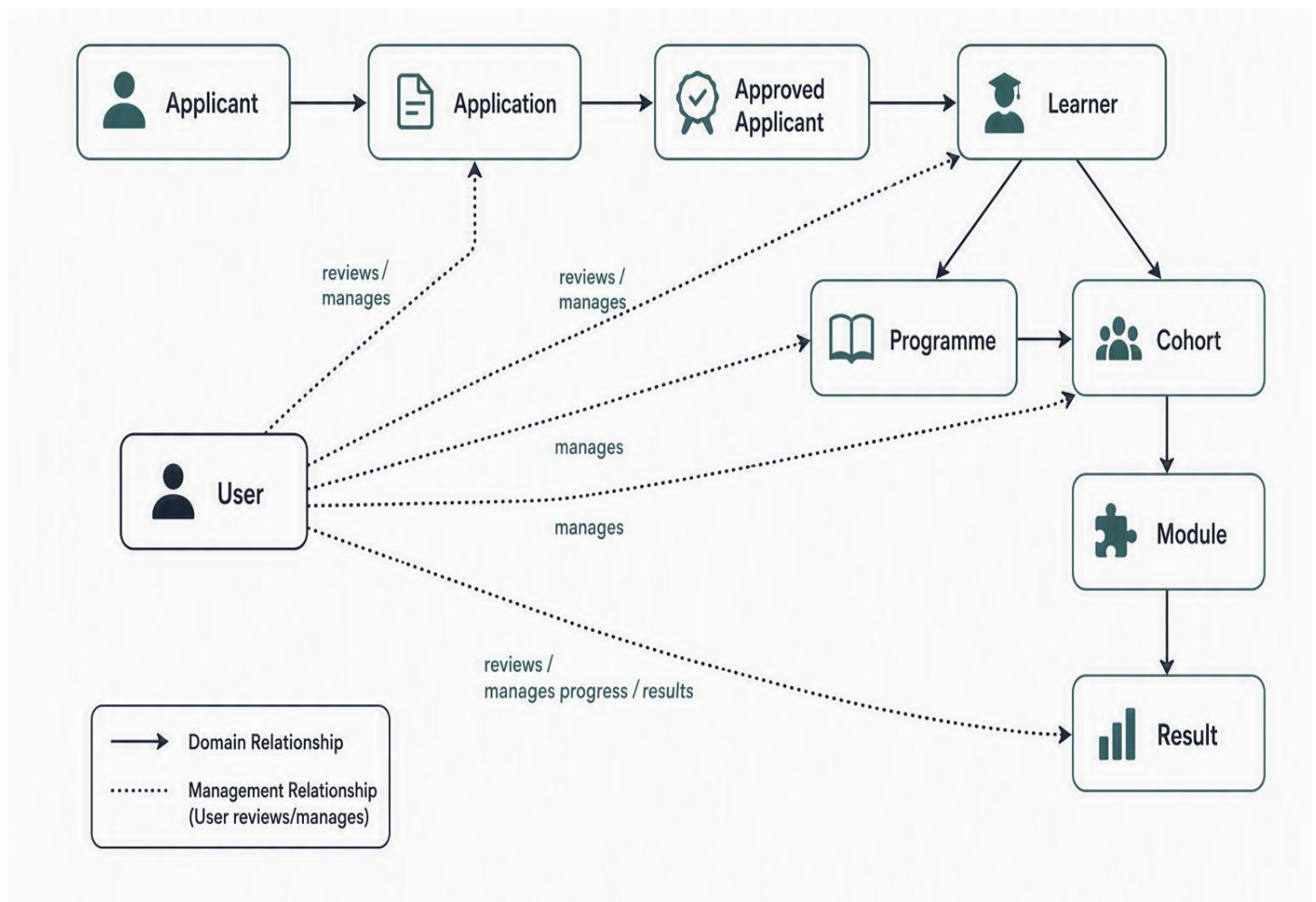
Programme

Cohort

Module

Result

Figure 6: basic Domain Relationship Diagram



11. Technical Direction

The implementation should use a conventional full-stack architecture appropriate for a small application.

Frontend

- HTML
- CSS
- JavaScript
- React

React will be used **alongside**, not instead of, the user's HTML/CSS/JavaScript fundamentals.

Backend

A suitable API-based backend will be selected during architecture design.

Database

- SQL relational database

Development

- VS Code
- Git
- GitHub
- GitHub Copilot

Design & modelling

- Figma -UI/UX
- PlantUML - UML
- diagrams.net - architecture diagrams

Testing

- Postman - API testing
- Appropriate automated/manual application testing

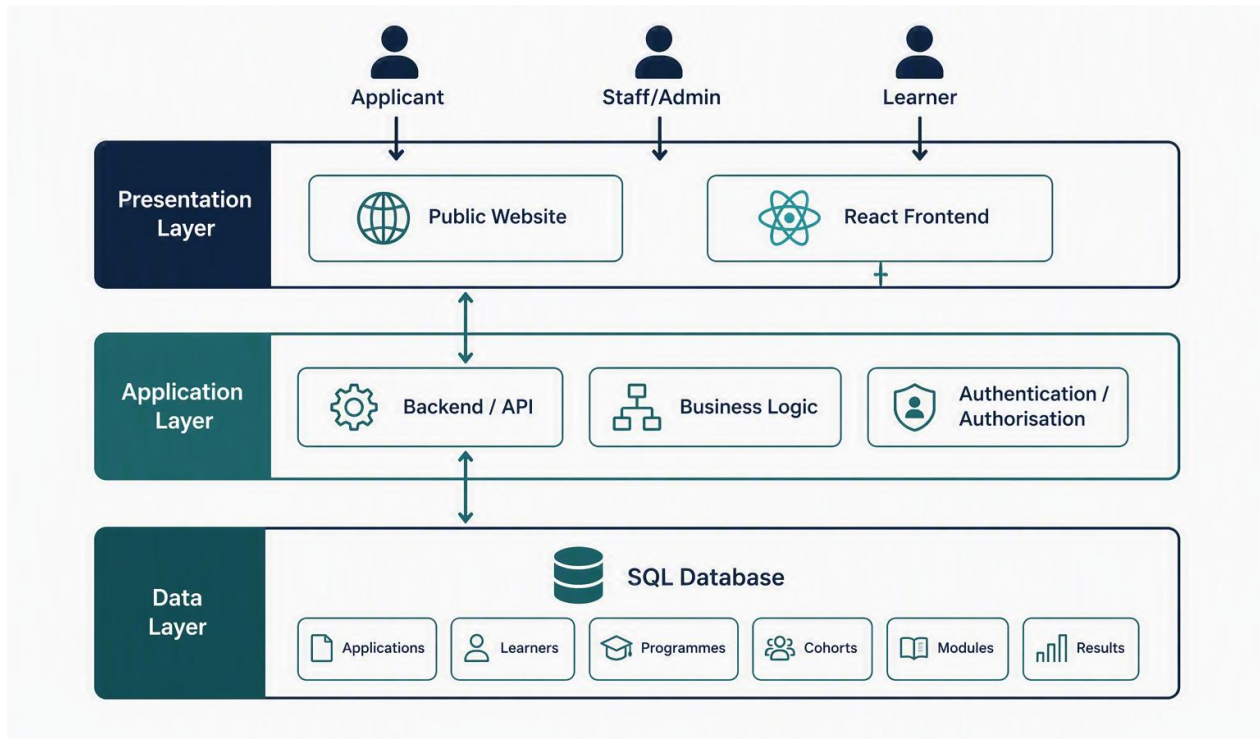
Cloud

AWS or Azure will be evaluated and selected based on:

- cost
- suitability
- simplicity

- available free/low-cost services
- security
- deployment requirements
- user's ability to implement and explain the solution

Figure 7: High Level System Architecture



12. Architecture Principles

The architecture should:

1. Keep V1 simple.
2. Avoid unnecessary technology.
3. Separate frontend, backend and database responsibilities.
4. Protect personal information.
5. Prefer managed services where appropriate.
6. Consider cost as part of technical decision-making.
7. Allow reasonable future growth.
8. Make important architectural decisions traceable to requirements.
9. Document significant trade-offs.
10. Prefer a solution that the developer can realistically build, test and maintain.

13. Constraints and Assumptions

- Zenith Platform is a small V1 project.
- The initial user population is expected to be relatively small.
- The platform will initially support one Zenith organisation.
- V1 does not require enterprise-scale infrastructure.
- The project should favour low-cost/free development and cloud services where practical.
- Personal information must be treated appropriately.
- Requirements may be refined during UML, architecture and implementation if a genuine technical or business issue is discovered.

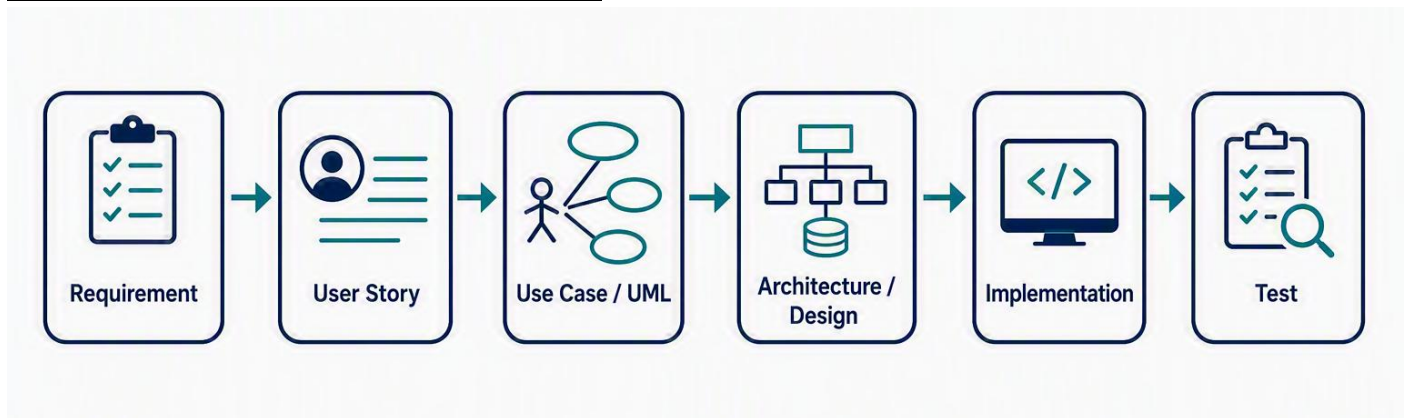
14. Acceptance Criteria

- A user can access the public website.
- A user can navigate the public pages.
- A user can view programme information.
- A user can complete and submit an application.
- Invalid/incomplete required information is rejected.
- A successful application is stored in the database.
- The applicant receives a submission confirmation.
- An authorised staff member can sign in.
- Staff can view submitted applications.
- Staff can view application details.
- Staff can approve or reject an application.
- An approved applicant can be associated with a learner record.
- Staff can view learner information.
- Learners can be associated with programmes/cohorts.
- Basic progress/results can be recorded where implemented.
- Unauthorised users cannot access protected information or functions.
- The application can be deployed to the selected cloud environment.

15. Requirements Traceability

Requirement → User Story → Use Case/UML → Design/Architecture → Implementation → Test

Figure 8: Requirements Traceability



This gives us evidence that the project was **engineered from requirements rather than simply coded**.

16. Project Artifacts

The V1 project should produce the following core artifacts:

1. **Requirements Document**
2. **User Stories**
3. **Use Case Diagram**
4. **Domain/Class Model**
5. **Key Sequence Diagram**
6. **ERD**
7. **Solution Architecture Diagram**
8. **Architecture Decisions & Trade-offs**
9. **Figma UI Design**
10. **Frontend**
11. **Backend**
12. **SQL Database**
13. **Authentication & Authorisation**
14. **Testing Evidence**
15. **Cloud Deployment**

17. Requirements Quality Rules

A requirement should be:

- **Clear**
- **Necessary**
- **Feasible**
- **Testable**
- **Consistent**
- **Traceable**
- **Prioritised**

Requirement language

Shall = mandatory V1 requirement.

Should = desirable requirement that may be deferred if necessary.

18. Change Control

If a requirement changes, record only:

Field	Description
Requirement ID	e.g. FR-06
Change	What changed
Reason	Why it changed
Impact	Effect on design/build
Decision	Accepted / Rejected / Deferred
Date	Date of decision

19. V1 Baseline

The Zenith Platform V1 will provide a **small end-to-end digital platform** that demonstrates:

Public information → Application → Application review → Approval → Learner management → Basic progress/results

